



Segundo avance del proyecto final

Materia:

Sistemas Distribuidos

Maestro:

Ramon Rene Palacio Cinco

Alumnos:

Martin Eliu Ayón Osuna

Jorge Omar Zazueta Huerta

Miguel Obed Navarro Mora

Jesús Alfredo Leonardo Elías

Descripción del Caso de Estudio

El caso consiste en un sistema para que ciudadanos presenten quejas ante PROFECO de forma digital. Los ciudadanos pueden registrar una queja, adjuntar evidencias y recibir notificaciones cuando un funcionario actualice el estado de su caso, sin necesidad de acudir a una oficina física.

La prueba de concepto no representa un sistema completamente terminado; su objetivo es demostrar que las comunicaciones entre los componentes del sistema funcionan correctamente, tal como fueron diseñadas.

Tecnologías y Justificación

Para la implementación del sistema se utilizó REST con JAX-RS para el registro, consulta y actualización de quejas y evidencias. Esta tecnología se eligió porque permite una comunicación clara basada en el modelo request-response, utilizando verbos HTTP como POST, GET y PUT, junto con respuestas en formato JSON y códigos de estado como 200, 201 y 404. Además, es un estándar ampliamente utilizado y fue visto en clase.

También se implementó SSE (Server-Sent Events) para notificar al ciudadano en tiempo real cuando su queja cambia de estado. Esta tecnología es adecuada porque permite una comunicación unidireccional desde el servidor hacia el cliente, evitando la necesidad de que el cliente realice consultas constantes. Se eligió sobre WebSockets debido a que no se requiere comunicación bidireccional.

Para la persistencia de datos se utilizó JDBC con PreparedStatement, lo cual permite ejecutar consultas SQL de manera segura y eficiente, evitando ataques como SQL Injection. Esta solución se integra con Agroal, que es el pool de conexiones utilizado por Quarkus.

En cuanto al manejo de archivos de evidencia, se utilizó Java NIO (Files), ya que permite leer y escribir archivos binarios de forma eficiente. Los archivos como imágenes o PDFs no se almacenan directamente en la base de datos, sino en el sistema de archivos.

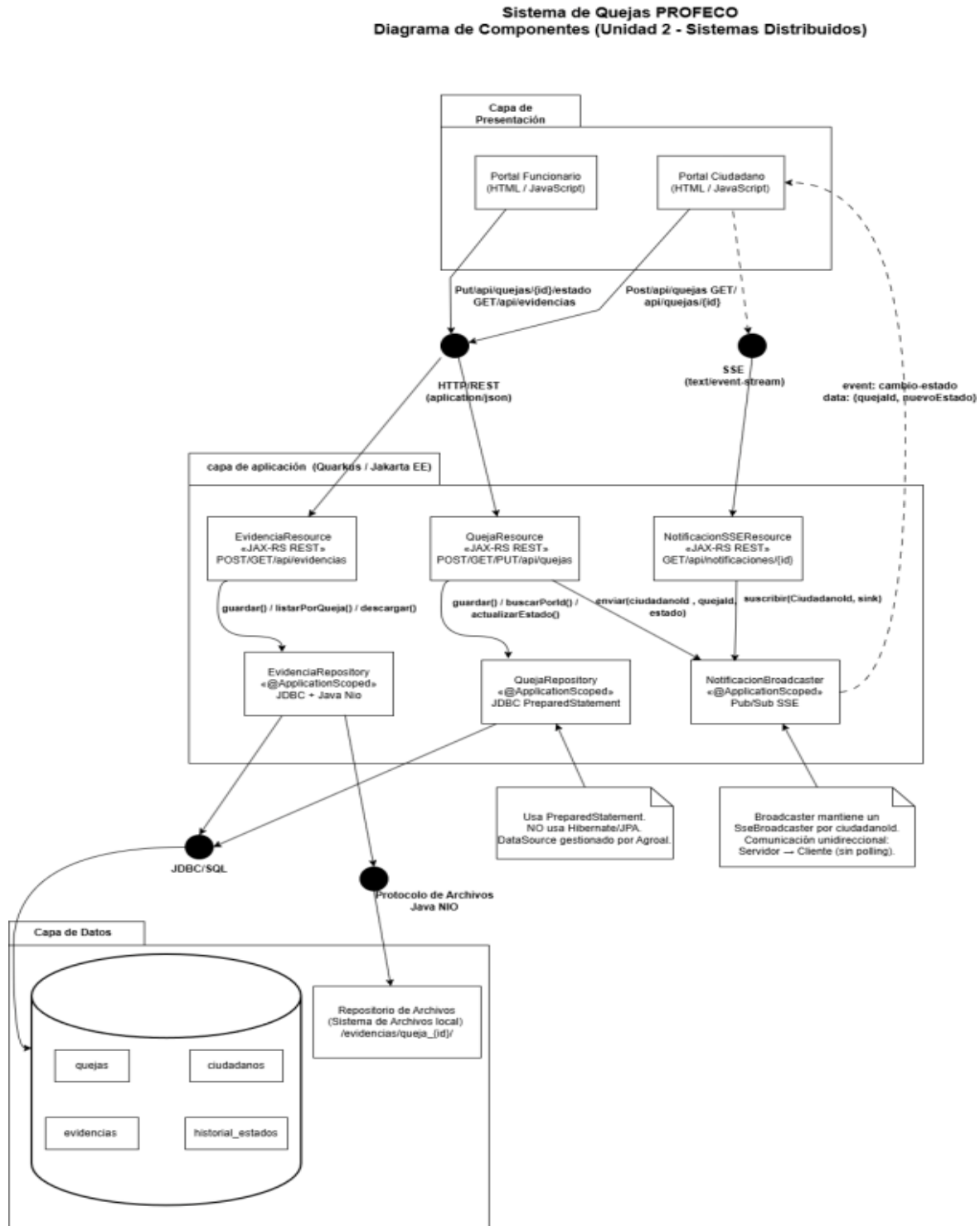
Para la serialización de datos se utilizó JSON con la librería Jackson, permitiendo convertir objetos Java a JSON y viceversa. Este formato es estándar en aplicaciones REST por su simplicidad e interoperabilidad.

El framework principal utilizado fue Quarkus 3.9 junto con Jakarta EE, ya que ofrece un entorno optimizado para microservicios, con arranque rápido, bajo consumo de memoria e integración de múltiples tecnologías en un solo entorno.

Finalmente, se utilizó PostgreSQL 15 como sistema de base de datos relacional, debido a su compatibilidad con JDBC y su capacidad para manejar datos estructurados como quejas, ciudadanos e historial de estados, priorizando la consistencia de la información.

Diagrama de Componentes

El diagrama de componentes muestra la organización interna del sistema, dividido en tres capas principales: presentación, aplicación y datos.



En la capa de presentación se encuentran el Portal Ciudadano y el Portal Funcionario. El Portal Ciudadano permite a los usuarios registrar quejas, consultar su estado y recibir notificaciones en tiempo real. Por otro lado, el Portal Funcionario permite a los empleados revisar y actualizar el estado de las quejas.

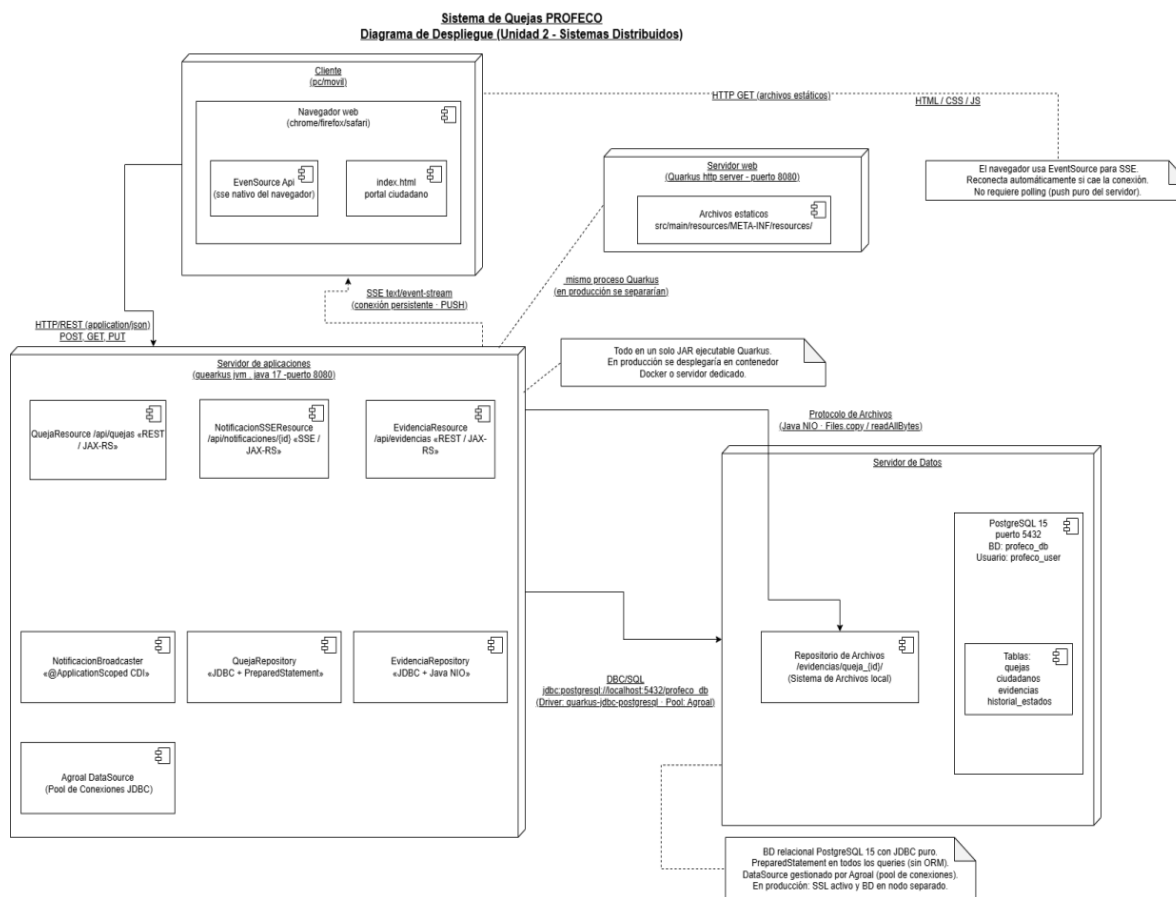
En la capa de aplicación, implementada con Quarkus, se encuentran varios componentes clave. El componente QuejaResource actúa como el endpoint REST principal, encargado de registrar, consultar y actualizar quejas. NotificacionSSEResource gestiona la conexión SSE con el cliente, manteniéndola abierta para enviar notificaciones. EvidenciaResource permite la subida y descarga de archivos de evidencia.

Además, existe un componente interno llamado NotificacionBroadcaster, el cual se encarga de enviar notificaciones a los clientes conectados mediante SSE cuando ocurre un cambio en el estado de una queja. Para el acceso a datos, se utilizan los componentes QuejaRepository y EvidenciaRepository, que interactúan con la base de datos mediante JDBC y con el sistema de archivos mediante Java NIO.

En la capa de datos se encuentra la base de datos PostgreSQL, que almacena la información de ciudadanos, quejas, evidencias e historial de estados. También se incluye un repositorio de archivos en disco donde se guardan las evidencias.

Diagrama de Despliegue

El diagrama de despliegue muestra cómo los componentes del sistema se distribuyen en distintos nodos físicos.



El nodo cliente corresponde a un dispositivo como una computadora o teléfono móvil que ejecuta un navegador web. Desde ahí se accede al sistema mediante HTTP y se reciben notificaciones en tiempo real utilizando SSE.

El servidor de aplicaciones, basado en Quarkus y ejecutándose en el puerto 8080, contiene toda la lógica del sistema. Este servidor recibe las solicitudes REST del cliente, mantiene las conexiones SSE activas y se comunica tanto con la base de datos como con el sistema de archivos.

El servidor de datos, donde se encuentra PostgreSQL ejecutándose en el puerto 5432, almacena toda la información estructurada del sistema. Además, en este nodo se encuentra el directorio donde se guardan los archivos de evidencia.

Flujo Principal de Comunicaciones

El flujo principal del sistema inicia cuando el ciudadano abre el portal en su navegador y establece una conexión con el endpoint SSE, la cual permanece abierta para recibir notificaciones.

Posteriormente, el ciudadano registra una queja enviando una solicitud POST al servidor. Este guarda la información en la base de datos y responde confirmando la operación. De manera opcional, el ciudadano puede subir archivos de evidencia, los cuales se almacenan tanto en la base de datos (metadatos) como en el sistema de archivos (contenido binario).

El funcionario consulta la queja mediante una solicitud GET y posteriormente puede actualizar su estado mediante una solicitud PUT. Cuando esto ocurre, el servidor actualiza la base de datos y envía una notificación al ciudadano a través de SSE.

Finalmente, el navegador del ciudadano recibe la notificación en tiempo real y actualiza la interfaz automáticamente, sin necesidad de realizar nuevas solicitudes.

Justificación de las Comunicaciones

Se eligió REST para la API principal debido a que las operaciones del sistema siguen el modelo natural de petición-respuesta. REST permite una comunicación sencilla, sin estado y ampliamente compatible con diferentes tecnologías.

Para las notificaciones se utilizó SSE, ya que el flujo de información es únicamente del servidor hacia el cliente. Esto evita la necesidad de realizar consultas constantes y reduce el tráfico innecesario en la red.

JDBC fue seleccionado para la comunicación con la base de datos debido a que permite un control directo sobre las consultas SQL, lo cual es útil para manejar operaciones específicas

como el historial de estados. Además, el uso de PreparedStatement mejora la seguridad del sistema.

No se utilizaron tecnologías como SOAP debido a su complejidad, ni WebSockets porque no se requiere comunicación bidireccional. Tampoco se utilizó polling, ya que generaría múltiples solicitudes innecesarias.

Drive con los diagramas con más calidad:

https://drive.google.com/drive/folders/1O2P2EseohfVCZAs_wcw-abzdmCbU-xGe